



## Trabalho 1 - Simulação paralela de autômatos celulares

Autômatos celulares são modelos computacionais formados por uma grade de células que evoluem ao longo do tempo de acordo com regras locais. Um dos exemplos mais conhecidos é o [Jogo da Vida](#), proposto por John Conway.

No Jogo da Vida tradicional, cada posição de uma grade bidimensional representa uma célula que pode estar viva ou morta. A cada geração, o estado de cada célula é atualizado de acordo com a quantidade de células vivas em sua vizinhança.

Neste trabalho será utilizada uma variação desse modelo. A grade possui dimensões  $L \times C$  e cada posição contém uma célula, que pode estar viva ou morta e uma regra de evolução associada àquela posição.

Diferentemente do Jogo da Vida tradicional, diferentes posições da grade poderão utilizar regras distintas. Dessa forma, duas células com o mesmo estado e a mesma quantidade de vizinhos vivos poderão apresentar comportamentos diferentes dependendo da regra associada à sua posição. A regra de cada posição permanece fixa durante toda a simulação. O estado das células, por outro lado, é atualizado a cada geração.

Cada célula possui até oito vizinhos: acima, abaixo, à esquerda, à direita e nas quatro diagonais. Nas bordas da matriz, são considerados apenas os vizinhos pertencentes à grade (ou seja, não é toroidal). A evolução ocorre de forma síncrona. Isso significa que todas as células da geração  $t+1$  devem ser calculadas utilizando exclusivamente os estados existentes na geração  $t$ .

### Regras de evolução

As regras utilizadas serão do tipo *Life-like* e serão representadas utilizando a notação B/S. A letra B, de *Birth*, indica com quantos vizinhos vivos uma célula atualmente morta passa a ficar viva. A letra S, de *Survival*, indica com quantos vizinhos vivos uma célula atualmente viva permanece viva.

Por exemplo, a regra clássica de Conway é representada por B3/S23. Isso significa que uma célula morta nasce se possuir exatamente 3 vizinhos vivos e uma célula viva sobrevive se possuir 2 ou 3 vizinhos vivos; nos demais casos, a célula estará morta na geração seguinte.

Outra regra possível é a HighLife (B36/S23), onde uma célula morta nasce se possuir 3 ou 6 vizinhos vivos e uma célula viva sobrevive se possuir 2 ou 3 vizinhos vivos. Portanto, uma célula morta com seis vizinhos vivos permaneceria morta em uma posição que utiliza B3/S23, mas passaria a ficar viva em uma posição que utiliza B36/S23.

Outras regras *Life-like* poderão ser utilizadas nas instâncias de validação. Para visualizar o funcionamento de diferentes regras e padrões iniciais, pode ser utilizado o seguinte simulador: <https://hmyzak.cz/software/gameoflife/index.html>

## Entrada

A primeira linha contém três números inteiros,  $L$ ,  $C$  e  $G$ , separados por espaços, onde  $L$  é o número de linhas da grade,  $C$  é o número de colunas e  $G$  é o número de gerações da simulação.

A linha seguinte contém o número de regras existentes  $R$ . As próximas  $R$  linhas contêm as regras disponíveis. Os identificadores das regras começam em 0 e correspondem à ordem em que aparecem na entrada.

Em seguida são fornecidas  $L$  linhas contendo a matriz de regras. Cada posição da matriz contém o identificador da regra utilizada naquela posição da grade.

Por fim, são fornecidas  $L$  linhas representando o estado inicial da matriz, onde  $x$  representa uma célula viva e um espaço em branco representa uma célula morta.

Cada linha da grade possui exatamente  $C$  posições. A entrada deverá ser lida pela entrada padrão (`stdin`).

## Saída

Após a execução das  $G$  gerações, o programa deverá imprimir o estado final da grade. A representação deverá utilizar  $x$  para células vivas e espaço em branco para células mortas.

A saída deverá conter exatamente  $L$  linhas, cada uma com  $C$  posições, e deverá ser escrita na saída padrão (`stdout`).

## Exemplo

| <u>Exemplo de <i>input</i> 1</u>                                                           | <u>Output esperado para o <i>input</i> 1</u> |
|--------------------------------------------------------------------------------------------|----------------------------------------------|
| <pre>5 5 3 1 B3/S23 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0    x   xx x  x</pre> | <pre>   x xx xx   x</pre>                    |

| <u>Exemplo de <i>input 2</i></u> | <u>Output esperado para o <i>input 2</i></u> |
|----------------------------------|----------------------------------------------|
| 6 8 5                            | x      x                                     |
| 2                                | x      x      x                              |
| B3/S23                           | x      x      x                              |
| B36/S23                          | x      x                                     |
| 0 0 0 0 1 1 1 1                  | x      x                                     |
| 0 0 0 0 1 1 1 1                  |                                              |
| 0 0 0 0 1 1 1 1                  |                                              |
| 0 0 0 0 1 1 1 1                  |                                              |
| 0 0 0 0 1 1 1 1                  |                                              |
| 0 0 0 0 1 1 1 1                  |                                              |
| 0 0 0 0 1 1 1 1                  |                                              |
| xx                               |                                              |
| xxxx                             |                                              |
| x      x                         |                                              |
| xxxx                             |                                              |
| xx                               |                                              |

## Desenvolvimento da solução

O problema **deverá** ser resolvido utilizando programação paralela em C, explorando múltiplos núcleos de processamento. Cada grupo deverá analisar o problema e propor a solução paralela que considerar mais adequada.

A versão sequencial (*life.c*), a ser utilizada para análise comparativa de desempenho, e instâncias de referência (*inputs* e *outputs* esperados) para verificação da implementação paralela podem ser obtidas em [life.zip](#). Para compilar e testar a versão sequencial disponibilizada, utilize:

```
$ make
$ ./run.sh                    # ou ./life < life-1.in
```

Teste também os demais arquivos de entrada disponíveis (*life-2.in*, *life-3.in* e *life-4.in*). Para cada caso de teste, verifique se a saída produzida pela versão paralela desenvolvida corresponde a saída esperada, contida no respectivo arquivo *life-\*.out*.

Além dos casos de teste disponibilizados, às instâncias de validação poderão apresentar características distintas, incluindo diferentes dimensões de grade, distribuições de células vivas, quantidades de regras e distribuições espaciais das regras. Algumas instâncias também poderão apresentar maior irregularidade ou desbalanceamento do trabalho. Certifique-se, portanto, de que a solução desenvolvida seja geral e não dependa de características específicas dos casos de teste fornecidos.

## Relatório e análise de desempenho

Além do código, cada grupo deverá entregar um relatório utilizando o [template](#) da Sociedade Brasileira de Computação (SBC).

O relatório deverá apresentar, de forma objetiva:

- a estratégia de paralelização adotada;
- as principais decisões de implementação;
- eventuais otimizações realizadas;
- os tempos de execução obtidos e o *speedup* observado;
- uma discussão dos resultados.

## Defesa, entrega e avaliação

O trabalho poderá ser realizado em grupos de até 3 participantes.

Deverão ser entregues pelo Moodle, em um arquivo .zip:

- código-fonte completo;
- Makefile atualizado para compilação do programa e arquivo README contendo instruções de compilação e execução;
- relatório utilizando o template SBC;
- *scripts* de teste e arquivos adicionais necessários para reproduzir a execução.

Todos os integrantes do grupo deverão estar identificados nos artefatos entregues.

Após a entrega, cada grupo realizará uma defesa de código em laboratório. Durante a defesa, os professores poderão solicitar explicações sobre a estratégia de paralelização, o funcionamento do código, as decisões de implementação e os resultados experimentais, além de solicitar a execução de casos específicos.

Todos os integrantes deverão compreender e ser capazes de explicar a solução entregue.

As soluções também serão executadas pelos professores no supercomputador Santos Dumont, utilizando condições equivalentes de execução.

Para participar da comparação de desempenho, a solução deverá produzir resultados corretos para as instâncias utilizadas na avaliação.

As soluções serão classificadas de acordo com o *speedup* obtido. As três equipes com melhor desempenho receberão um bônus de 0,5 ponto na nota do trabalho (estando limitada a nota final do trabalho a 10,0 pontos), desde que a solução esteja correta e que os integrantes demonstrem domínio da implementação durante a defesa.